STA 556 — Homework 3

Reproducibility & Matrix Computation

Release: Thursday, October 29, 2026
Due: Thursday, November 5, 2026
Total: 40 points
Primary coverage: Sections 4.2 and 5.1

Goal

Create a **reproducible Quarto report** that uses NumPy matrix operations to fit and diagnose a small linear-model problem.

This homework deliberately stops **before simulation and resampling**.

You will choose **one of three provided synthetic datasets**. Do not create your own dataset.

Dataset options

Choose **one** dataset.

Option A — Student performance

File:

```
data/student_performance.csv
```

Use:

```
response: final_score

predictors:
study_hours_per_week
attendance_pct
prior_score
```

Option B — Housing

File:

```
data/housing.csv
```

Use:

```
response: price_thousands

predictors:
size_sqft
bedrooms
age_years
```

Option C — Watershed conditions

File:

```
data/watershed.csv
```

Use:

```
response: streamflow_index

predictors:
snowpack_pct_normal
precip_mm
mean_temp_c
```

All three datasets are synthetic and are intended only for this homework. Do not make causal claims.

Part 1 — Reproducible report and data description (8 points)

Create:

`reports/homework3.qmd`

and render it to HTML.

Your report must successfully run from beginning to end using:

```
quarto render reports/homework3.qmd
```

from the repository root.

Your report must contain the following subsections.

1.1 Introduction

In **one short paragraph**:

- identify which dataset you selected;
- identify the response variable;
- identify the three predictors;
- state that the goal is to fit and diagnose a linear model using matrix computation.

1.2 Data inspection and validation

Load the CSV using a **relative path**.

Report and inspect:

```
df.head()
df.shape
df.dtypes
df.isna().sum()
df["id"].is_unique
```

Write **2–4 sentences** stating:

- the number of observations;
- the number of variables;
- whether missing values are present;
- whether IDs are unique.

1.3 Reproducibility requirements

Your report must:

- execute in order from a clean session;
- contain no hidden notebook state;
- contain no manually copied numerical results;
- use relative paths;
- generate all tables and numerical results from code;
- use the provided dataset without manually entering values.

Part 2 — Matrix setup and standardization (8 points)

Use the **three predictor columns specified for your chosen dataset**.

Convert the predictors and response to NumPy arrays.

```
predictors = df[
    [
        # your three predictor columns
    ]
].to_numpy()

y = df["response_column"].to_numpy()
```

Standardize the predictors

Using NumPy only:

```
means = predictors.mean(axis=0)
sds = predictors.std(axis=0)

Z = (
    predictors - means
) / sds
```

Do **not** use `sklearn` for standardization.

Verify approximately:

```
Z.mean(axis=0)
Z.std(axis=0)
```

Construct the design matrix

Add an intercept column:

```
X = np.column_stack([
    np.ones(len(Z)),
    Z
])
```

Report:

- shape of `predictors`;
- shape of `Z`;
- shape of `X`;
- shape of `y`;
- matrix rank of `X`;
- number of predictors excluding the intercept.

Written response

In **3–5 sentences**, explain:

1. why `X` and `y` must have compatible shapes;
2. why the intercept column is included;
3. what standardizing the predictors accomplished;
4. why broadcasting makes this standardization possible without a Python loop.

Part 3 — Least squares (8 points)

Fit the model using:

```
beta_hat, *_ = np.linalg.lstsq(
    X,
    y,
    rcond=None
)
```

Report the estimated coefficients.

Because the predictors were standardized, identify:

- the intercept;
- the coefficient for each standardized predictor;
- which predictor has the largest coefficient in absolute value.

Calculate fitted values:

```
fitted = X @ beta_hat
```

and residuals:

```
residuals = y - fitted
```

Calculate and report:

```
residuals.mean()
```

and:

```
rmse = np.sqrt(
    np.mean(
        residuals ** 2
    )
)
```

Written response

In **3–5 sentences**, explain:

1. what `X @ beta_hat` represents;
2. why the residual mean should be near zero when an intercept is present;
3. what RMSE summarizes;
4. the difference between:

```
X * beta_hat
```

and:

```
X @ beta_hat
```

Part 4 — QR, SVD, and conditioning (10 points)

QR decomposition

Compute:

```
Q, R = np.linalg.qr(X)
```

Report the shapes of `Q` and `R`.

Verify:

```
np.allclose(
    Q @ R,
    X
)
```

and:

```
np.allclose(
    Q.T @ Q,
    np.eye(Q.shape[1])
)
```

SVD

Compute:

```
U, s, Vt = np.linalg.svd(
    X,
    full_matrices=False
)
```

Report:

- shapes of `U`, `s`, and `Vt`;
- singular values.

Reconstruct `X`:

```
X_reconstructed = (
    U
    @ np.diag(s)
    @ Vt
)
```

Verify:

```
np.allclose(
    X_reconstructed,
    X
)
```

Rank and conditioning

Report:

```
np.linalg.matrix_rank(X)
np.linalg.cond(X)
```

Also construct the **raw, unstandardized design matrix**:

```
X_raw = np.column_stack([
    np.ones(len(predictors)),
    predictors
])
```

and report:

```
np.linalg.cond(X_raw)
```

Written response

In **5–8 sentences**, explain:

1. what `Q @ R ≈ X` verifies;
2. what `Q.T @ Q ≈ I` verifies;
3. what the singular values tell you about strong and weak directions;
4. what a large condition number suggests;
5. whether the condition number changed after standardization;
6. why measurement scale can affect the raw condition number;
7. whether the standardized design matrix shows evidence of severe near-redundancy among predictors.

Part 5 — Reproducibility and numerical-computing reflection (6 points)

Write **250–400 words** addressing all of the following:

1. Why does clean execution matter for a computational report?
2. Why should reusable code not depend on hidden notebook state?
3. Why is `np.linalg.lstsq()` preferable to manually computing $(X^TX)^{-1}X^Ty$ in ordinary numerical work?
4. Why are checks such as `np.allclose()` preferable to exact equality for QR and SVD reconstruction?
5. What did the condition-number comparison teach you about scale and numerical diagnostics?
6. Name one reproducibility practice from this homework that you will carry into the final project.

Final reproducibility check

Before submission:

1. save all files;
2. restart any active notebook kernels;
3. from the repository root run:

```
quarto render reports/homework3.qmd
```

4. confirm the report renders without manual intervention;
5. run:

```
git status
```

6. commit and push the final version.

Grading rubric

Area	Points
Reproducible report and data description	8
Matrix setup and standardization	8
Least squares	8
QR/SVD diagnostics and conditioning	10
Reflection	6
Total	**40**

What to submit

Submit the link to your GitHub repository in Canvas.

The repository must contain:

```
reports/homework3.qmd
reports/homework3.html
```

and the selected dataset in the provided `data/` directory.

The instructor (Dr Ben) should be able to reproduce the HTML report by running:

```
quarto render reports/homework3.qmd
```